
Artefatos de Teste

Casos de teste, rastreabilidade e classes de equivalência



Prof. Dr. João Choma

Teste e Qualidade de Software

github.com/JoaoChoma

Semana 02

Artefatos tornam o teste verificável



Um teste profissional deixa respostas registradas:

- o que e por que será testado;
- como e com quais dados o teste será executado;
- qual comportamento era esperado;
- o que realmente aconteceu;
- qual evidência sustenta a conclusão.

Ideia central

Os **artefatos de teste** organizam essas respostas.



- Distinguir os principais artefatos de teste.
- Explicar como eles se conectam.
- Reconhecer o caso como elo entre planejamento e execução.
- Escrever casos claros, reproduzíveis e rastreáveis.
- Derivar casos a partir de requisitos.
- Criar classes de equivalência válidas e inválidas.
- Selecionar representantes sem desperdiçar testes.



Artefatos são documentos, registros e entregas produzidos durante o teste.

- Orientam o trabalho antes da execução.
- Tornam a execução repetível.
- Registram resultados e evidências.
- Comunicam riscos e defeitos.
- Demonstram cobertura dos requisitos.
- Apoiam decisões sobre a qualidade.



Uma afirmação insuficiente

“Eu testei e parece estar funcionando.”

Ainda faltaria saber:

- qual versão, cenário e dados foram testados;
- qual resultado era esperado;
- quais requisitos foram cobertos;
- como reproduzir uma eventual falha.

Um teste sem registro é difícil de revisar e repetir.

O conjunto de artefatos



Artefato	Pergunta principal
Plano de teste	O que, como, quando e com quais recursos?
Caso de teste	Qual situação específica será verificada?
Roteiro	Qual sequência operacional será seguida?
Dados de teste	Quais valores e estados serão utilizados?
Registro de execução	O que aconteceu durante a execução?
Evidência	Como demonstramos o resultado?



Relatório de defeito explica e permite reproduzir uma falha.

Relatório de teste consolida a situação da atividade.

Matriz de rastreabilidade conecta requisitos, casos, execuções e defeitos.

Cada documento tem uma função própria, mas todos devem contar a mesma história.



Requisito

- > Plano de teste
 - > Caso + dados de teste
 - > Execucao + evidencias
 - > Resultado
 - > Defeito, se houver divergencia
 - > Relatorio de teste

A matriz de rastreabilidade conecta toda a cadeia.



REQ-07 — Idade do paciente

O sistema deve aceitar pacientes com idade entre 18 e 65 anos, inclusive. Valores fora do intervalo devem ser rejeitados com mensagem explicativa.

O exemplo permite discutir regra, dados válidos e inválidos, resultados, limites, rastreabilidade e defeitos.

Plano, dados e ambiente



- Objetivos, riscos e escopo.
- Níveis e tipos de teste.
- Recursos, ambientes e responsabilidades.
- Cronograma e marcos.
- Critérios de entrada, saída, suspensão e retomada.
- Artefatos que serão produzidos.

O plano orienta o conjunto; não substitui os casos.



- ☐ Projeto, versão e responsáveis identificados.
- ☐ Objetivos, riscos, escopo e exclusões explícitos.
- ☐ Estratégia, níveis e tipos de teste definidos.
- ☐ Ambiente, ferramentas, dados e recursos previstos.
- ☐ Cronograma, marcos e responsabilidades acordados.
- ☐ Critérios de entrada, saída, suspensão e retomada.
- ☐ Processo de defeitos, comunicação e entregáveis.
- ☐ Aprovação e controle de mudanças registrados.

O checklist reduz omissões; o risco define a profundidade.



Plano de Teste - Sistema de Gerenciamento de Tarefas

1. Introdução

Este plano de teste descreve a abordagem para testar o Sistema de Gerenciamento de Tarefas, que tem como objetivo permitir que os usuários criem, acompanhem e gerenciem suas tarefas diárias de forma eficiente. O objetivo deste plano é garantir a qualidade e confiabilidade do software antes do lançamento.

2. Objetivos

Os objetivos do teste são:

- Verificar se todas as funcionalidades do sistema de gerenciamento de tarefas estão implementadas corretamente.
- Validar se o sistema atende aos requisitos funcionais e não funcionais especificados.
- Identificar e corrigir defeitos encontrados durante o teste.

3. Escopo

O teste abrangerá todas as funcionalidades principais do Sistema de Gerenciamento de Tarefas, incluindo:

- Criação e edição de tarefas.
- Atribuição de tarefas a usuários.
- Acompanhamento do progresso das tarefas.
- Classificação e filtragem de tarefas.
- Notificações e lembretes de tarefas.

4. Estratégia de Teste

A estratégia de teste inclui:

- Testes unitários realizados pelos desenvolvedores para verificar as funcionalidades individualmente.
- Testes de integração para garantir que as diferentes partes do sistema funcionem em conjunto.
- Testes de sistema para verificar o funcionamento do sistema como um todo.
- Testes de aceitação realizados pelos usuários finais para validar o sistema em um ambiente de produção simulado.

5. Casos de Teste

Serão criados casos de teste para cada funcionalidade do sistema, abrangendo cenários positivos e negativos, bem como casos de teste de limite e estresse.

Exemplo de Caso de Teste:

Caso de Teste 1 - Criar Tarefa

- Descrição: Verificar se é possível criar uma nova tarefa no sistema.

- Pré-condições: O usuário está logado no sistema.

- Passos:

1. Acessar a página de criação de tarefas.

2. Preencher o formulário com os dados da nova tarefa (título, descrição, data, prioridade, etc.).

O modelo em docs/Plano de Teste.docx organiza:

- contexto e objetivos;
- escopo e estratégia;
- ambiente e recursos;
- cronograma e critérios;
- riscos e responsabilidades;
- comunicação e aprovação.



Dentro do escopo

- campos obrigatórios;
- validação da idade;
- persistência;
- mensagens ao usuário.

Fora desta rodada

- desempenho em escala;
- integração com convênios;
- mensagens por e-mail.

Definir limites reduz expectativas contraditórias.



“Informar uma idade válida” é uma orientação vaga.

```
nome: Ana Silva  
idade: 30  
CPF: 529.982.247-25  
e-mail: ana.teste@example.com
```

Dados devem considerar validade, privacidade, estado inicial e reutilização.



- 1 Acessar o ambiente de homologação.
- 2 Autenticar-se como recepcionista.
- 3 Abrir **Pacientes** > **Novo cadastro**.
- 4 Preencher os dados indicados no caso.
- 5 Confirmar a operação.
- 6 Registrar resultado e evidência.

Separar o roteiro evita repetir instruções extensas.



Adequadas

- usuário autenticado;
- ambiente disponível;
- CPF não cadastrado;
- base na versão definida.

Evite

- esconder passos;
- depender de dados instáveis;
- “sistema funcionando normalmente”.

A pré-condição precisa ser verificável.

Execução, evidência e defeito



Para cada execução, registre:

- caso, versão, data, ambiente e executor;
- dados efetivamente utilizados;
- resultado obtido;
- status: passou, falhou ou bloqueado;
- evidências associadas;
- defeito relacionado, quando aplicável.

O resultado real só deve ser preenchido após a execução.



Uma boa evidência:

- mostra a interface ou o log que sustenta a conclusão;
- identifica o caso, a execução e a versão;
- preserva os valores relevantes;
- evita dados pessoais ou segredos;
- pode ser localizada por outra pessoa.

Uma captura sem contexto pode não provar nada.



```
DEF-014 - Cadastro aceita paciente menor de 18 anos
Ambiente: homologacao / build 2026.08.04
Pre-condicao: CPF nao cadastrado
Passos: executar CT-07B com idade 17
Esperado: rejeitar e explicar o intervalo
Obtido: cadastro salvo; mensagem de sucesso
Severidade: alta
```

O defeito referencia o caso; não precisa repetir todo o planejamento.



- Casos planejados, executados, aprovados e falhos.
- Testes bloqueados e seus motivos.
- Requisitos cobertos e lacunas.
- Defeitos por severidade e situação.
- Riscos residuais e recomendação para a entrega.

Atenção

Contagens ajudam, mas a decisão depende do risco das falhas.

Rastreabilidade

A matriz mostra se os requisitos foram testados



Req.	Caso	Execução	Resultado	Defeito
REQ-07	CT-07A — 30	EX-021	Passou	—
REQ-07	CT-07B — 17	EX-022	Falhou	DEF-014
REQ-07	CT-07C — 66	EX-023	Passou	—

A matriz revela lacunas, impacto de mudanças e falhas relacionadas.



- ☐ Todos os requisitos em escopo possuem ID.
- ☐ Cada requisito aponta para pelo menos um caso.
- ☐ Positivos, negativos e limites relevantes estão cobertos.
- ☐ Execução e status podem ser localizados.
- ☐ Falhas apontam para defeitos e evidências.
- ☐ Lacunas de cobertura estão sinalizadas.
- ☐ Mudanças foram propagadas para os casos.
- ☐ Versão, responsável e atualização estão registrados.

A matriz é mantida durante o projeto, não somente no final.



Req.	Descrição	Caso	Cenário	Exec.	Status	Defeito	Evid.
REQ-07	aceitar 18–65	CT-07A	idade 30	EX-021	passou	—	EVD-021
REQ-07	rejeitar fora	CT-07B	idade 17	EX-022	falhou	DEF-014	EVD-022
REQ-07	rejeitar fora	CT-07C	idade 66	EX-023	passou	—	EVD-023

O modelo docs/Matriz de Rastreabilidade.docx associa requisitos e casos. Na execução, acrescente status, defeito e evidência para acompanhar o ciclo completo.



O caso conecta:

- requisito e risco;
- dados e roteiro;
- resultado esperado;
- execução e evidência;
- defeito e rastreabilidade.

Consequência

Escrever bons casos melhora todos os artefatos ao redor.

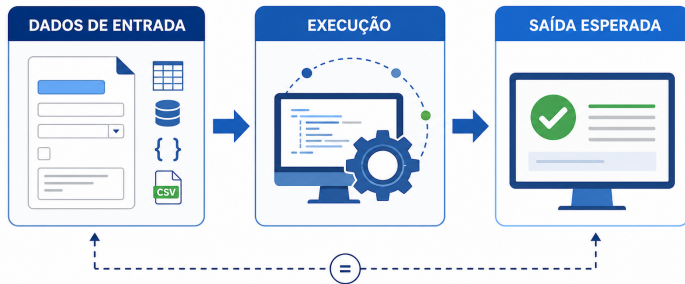
Aula prática: casos de teste



Um caso descreve uma situação controlada. Outra pessoa deve conseguir:

- 1 preparar o mesmo estado;
- 2 usar os mesmos dados;
- 3 realizar as mesmas ações;
- 4 observar o mesmo ponto de verificação;
- 5 concluir se passou ou falhou.

Clareza e repetibilidade importam mais que volume de texto.



O resultado obtido será comparado com uma saída esperada definida **antes** da execução.



Campo	Função
ID e título	Identificar e comunicar a condição.
Requisito e objetivo	Manter origem e comportamento em foco.
Pré-condições	Definir o estado inicial.
Dados de entrada	Informar valores concretos.
Passos	Orientar as ações necessárias.
Resultado esperado	Definir o comportamento correto.
Prioridade	Orientar a ordem de execução.

Resultado real, status e evidência pertencem à execução.



- ☐ ID, título, requisito e prioridade identificados.
- ☐ Objetivo ou condição principal delimitada.
- ☐ Pré-condições verificáveis e estado controlado.
- ☐ Dados concretos e reproduzíveis.
- ☐ Passos suficientes e em ordem.
- ☐ Resultado esperado observável e prévio à execução.
- ☐ Pós-condições e efeitos persistentes considerados.
- ☐ Espaço para real, status, evidência e defeito.
- ☐ Versão ou responsável pela atualização registrado.

Caso de Teste - Funcionalidade de Login

Descrição: Verificar se a funcionalidade de login permite que os usuários acessem o sistema com credenciais válidas e impede o acesso com credenciais inválidas.

Pré-condições: O sistema está em execução e a página de login está acessível.

Passos:

1. Acessar a página de login do sistema.
2. Preencher o campo "Nome de usuário" com um nome de usuário válido.
3. Preencher o campo "Senha" com uma senha válida.
4. Clicar no botão "Entrar".

Cenário 1 - Login bem-sucedido

- Dados de Teste:
- Nome de usuário: "usuario123"
- Senha: "senha123"

- Resultado Esperado:

- O sistema redireciona o usuário para a página inicial após o login.

Cenário 2 - Nome de usuário inválido

- Dados de Teste:
- Nome de usuário: "usuario_invalido"
- Senha: "senha123"

- Resultado Esperado:

- O sistema exibe uma mensagem de erro indicando que o nome de usuário é inválido.
- O usuário permanece na página de login.

Cenário 3 - Senha inválida

- Dados de Teste:
- Nome de usuário: "usuario123"
- Senha: "senha_invalida"

- Resultado Esperado:

- O sistema exibe uma mensagem de erro indicando que a senha é inválida.
- O usuário permanece na página de login.

Cenário 4 - Nome de usuário e senha em branco

- Dados de Teste:
- Nome de usuário: [campo em branco]
- Senha: [campo em branco]

- Resultado Esperado:

- O sistema exibe mensagens de erro indicando que os campos "Nome de usuário" e "Senha" são obrigatórios.

O exemplo em docs/Caso de Teste.docx apresenta:

- funcionalidade e pré-condições;
- passos de execução;
- cenários de login;
- dados de teste;
- resultados esperados.

Acrescente ID, requisito, prioridade e dados da execução.



Fracos

- Testar idade
- Teste do cadastro
- Verificar paciente

Melhores

- Aceitar idade permitida.
- Rejeitar idade inferior a 18.
- Rejeitar idade superior a 65.

O título ajuda a entender a cobertura sem abrir o caso inteiro.



Campo	Conteúdo
ID / requisito	CT-07A / REQ-07
Título	Aceitar paciente com idade válida
Pré-condições	Recepcionista autenticado; CPF não cadastrado
Entrada	Ana Silva; idade 30; demais campos válidos
Passos	Abrir cadastro; preencher dados; confirmar
Esperado	Salvar uma vez; criar ID; exibir sucesso
Prioridade	Alta

Verifique efeitos observáveis, não apenas uma mensagem.



Campo	Conteúdo
ID / requisito	CT-07B / REQ-07
Título	Rejeitar idade inferior ao mínimo
Pré-condições	Recepcionista autenticado; CPF não cadastrado
Entrada	Bruno Lima; idade 17; demais campos válidos
Passos	Abrir cadastro; preencher dados; confirmar
Esperado	Não persistir; destacar idade; informar 18 a 65
Prioridade	Alta

Em testes negativos, confirme que nenhuma alteração indevida ocorreu.



Vago

O sistema deve funcionar corretamente.

Observável

O cadastro não é persistido; a idade permanece preenchida e destacada; a mensagem “Informe uma idade entre 18 e 65 anos” é exibida.

Informe **o que muda**, **o que não muda**, **onde observar** e **qual valor deve aparecer**.



Insuficiente

“Cadastrar paciente.”

Excessivo

Descrever cada clique e tecla.

Nível adequado

- 1 Abrir **Pacientes** > **Novo**.
- 2 Preencher os dados indicados.
- 3 Selecionar **Salvar**.
- 4 Consultar pelo CPF.



Misturar idade, CPF, e-mail, senha e permissões provoca:

- causa da falha pouco clara;
- resultado esperado ambíguo;
- manutenção difícil;
- rastreabilidade imprecisa.

Combine condições somente quando a interação for parte do risco.



- 1 Derive de requisito, regra ou risco.
- 2 Use IDs estáveis.
- 3 Informe dados concretos.
- 4 Controle o estado inicial.
- 5 Use verbos de ação.
- 6 Escreva o esperado antes de executar.
- 7 Verifique interface, dados e integrações.
- 8 Mantenha independência quando possível.
- 9 Atualize quando o requisito mudar.
- 10 Faça revisão por outra pessoa.



- A origem está identificada e a condição aparece no título?
- As pré-condições são verificáveis?
- Os dados são concretos e reproduzíveis?
- Outra pessoa consegue seguir os passos?
- O esperado é objetivo e observável?
- O caso possui uma intenção principal?
- Está claro qual evidência deve ser coletada?

Se alguma resposta for “não”, o caso ainda pode melhorar.

Classes de equivalência



Uma **classe de equivalência** reúne entradas que esperamos que o sistema trate da mesma maneira.

- classes **válidas**, aceitas pela regra;
- classes **inválidas**, rejeitadas pela regra.

A técnica reduz testes sem abandonar comportamentos distintos.



- 1 Escolha uma condição de entrada.
- 2 Escreva a regra com precisão.
- 3 Divida o domínio por comportamento esperado.
- 4 Classifique cada partição como válida ou inválida.
- 5 Confirme que não há sobreposição.
- 6 Selecione representantes.
- 7 Transforme-os em casos de teste.

Exemplo 1: idade entre 18 e 65 anos



Classe	Tipo	Valor	Esperado
$\text{idade} < 18$	Inválida	10	Rejeitar
$18 \leq \text{idade} \leq 65$	Válida	30	Aceitar
$\text{idade} > 65$	Inválida	70	Rejeitar

Três representantes cobrem três comportamentos diferentes.



Região	Valores interessantes
Limite inferior	17, 18 , 19
Limite superior	64, 65 , 66

- equivalência seleciona comportamentos diferentes;
- valor-limite investiga as bordas das partições.

Se o risco for alto, use as duas técnicas.

Exemplo 2: quantidade obrigatória de 1 a 99



Classe	Tipo	Exemplo
Inteiro entre 1 e 99	Válida	25
Inteiro menor que 1	Inválida	0
Inteiro maior que 99	Inválida	100
Número não inteiro	Inválida	2,5
Valor não numérico	Inválida	abc
Valor ausente	Inválida	vazio



- formato aceito: `ana@example.com`;
- ausência de @: `ana.example.com`;
- domínio ausente: `ana@`;
- parte local ausente: `@example.com`;
- espaços proibidos: `ana silva@example.com`;
- valor vazio, quando obrigatório.

Divida classes quando acionarem validações ou riscos diferentes.



Regra: tipo deve ser consulta, retorno ou exame.

- Cada opção pode ser válida se produzir fluxos diferentes.
- urgência, fora da lista, forma classe inválida.
- Valor vazio forma outra classe quando obrigatório.
- Maiúsculas e espaços seguem a regra especificada.

Nem toda classe é um intervalo numérico.



- Considerar apenas a classe válida.
- Esquecer vazio, nulo, formato e tipo.
- Criar classes sobrepostas.
- Testar a fronteira e ignorar o interior.
- Misturar condições independentes.
- Supor equivalência sem conhecer o comportamento.
- Criar classes sem diferença real de tratamento.



- Escolha um valor típico do interior.
- Acrescente limites quando houver fronteiras.
- Prefira valores fáceis de reconhecer.
- Evite colisões com registros existentes.
- Documente por que o valor representa a classe.
- Ao testar uma classe inválida, mantenha as demais condições válidas.
- Aumente a amostra quando o risco justificar.



```
CT-07A - aceitar idade tipica valida: 30
CT-07B - rejeitar idade abaixo: 10
CT-07C - rejeitar idade acima: 70
CT-07D - aceitar limite inferior: 18
CT-07E - aceitar limite superior: 65
CT-07F - rejeitar imediatamente abaixo: 17
CT-07G - rejeitar imediatamente acima: 66
```

A classe orienta a seleção; o caso registra a execução completa.

Atividade prática



Regra da senha

De 8 a 20 caracteres, com uma maiúscula, uma minúscula e um algarismo. Espaços não são permitidos.

Em grupos:

- 1 identifique classes e limites;
- 2 escreva três casos completos;
- 3 associe cada caso à regra;
- 4 indique a evidência;
- 5 descreva um possível defeito.



- válida, de 8 a 20;
- menos de 8;
- mais de 20;
- sem maiúscula;
- sem minúscula.

- sem algarismo;
- contendo espaço;
- valor vazio.

Limites: 7, 8, 9, 19, 20 e 21.

Válida: Teste123



Troque um caso e execute apenas com o que está escrito. Registre:

- informação ausente ou passo ambíguo;
- dado difícil de reproduzir;
- resultado esperado não observável;
- requisito ou classe sem cobertura;
- melhoria sugerida.

Se depende de explicação oral, ainda não está pronto.

Síntese



O requisito define o comportamento.

O plano define a estratégia.

As classes orientam os dados.

O caso define a verificacao.

O roteiro orienta a execucao.

A evidencia demonstra o resultado.

O defeito registra a divergencia.

O relatorio apoia a decisao.

Rastreabilidade permite avaliar mudanças e falhas com segurança.



- Confirme o requisito e o risco.
- Revise classes válidas, inválidas e limites.
- Prepare dados e estado inicial.
- Escreva resultados observáveis.
- Defina evidências úteis.
- Execute sem alterar a expectativa.
- Registre resultado, status e defeitos.
- Atualize a rastreabilidade.



- SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed., 2019.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed., 2016.
- PFLEEGER, Shari Lawrence. *Engenharia de Software: teoria e prática*. 2. ed., 2004.

Imagem do fluxo gerada especificamente para esta aula.